From Hardware to Software: End to End Leakage Verification

ANONYMOUS AUTHOR(S)

TBD

Additional Key Words and Phrases: Hardware, Side-Channels, Leakage

1 Introduction

2 Overview

2.1 Example: Adder

Hardware Circuits. Our goal is to give high-level leakage descriptions of hardware circuits. A circuit is a function with the following type, where `S` is the type of its state, `I` is the type of its inputs and `O` is the type of its outputs.

```
1 type Circ S I O = ((S, I) -> (S, O), S)
```

A circuit with input type `I` and output type `O` can be turned into a function from sequences of inputs to sequences of outputs, that is, a function of type `[I] -> [O]`. This function, which we call the transducer of the circuit represents the circuits input/output behavior in hardware.

Adder Implementation. Our first example is a hardware implementation of an adder. The adder outputs a pending result, if present (line 2). If the first input is `0` (line 3), the adder takes a fast path, and outputs the result – in this case, the second input – right away. If not, it stores the intermediate result in the circuit state (line 4). Otherwise, the circuit outputs Nothing, and stores no new intermediate result (line 5).

```
1 add :: Circ (Maybe Int) (Maybe (Int, Int)) (Maybe Int)
2 add (Just res) _      = (Nothing, Just res)
3 add _ (Just (0, b))   = (Nothing, Just b)
4 add _ (Just (a, b))   = (Just (a + b), Nothing)
5 add Nothing _         = (Nothing, Nothing)
```

Running the Circuit. We can produce the circuit's transducer using function `mealy` below. Mealy takes as input a circuit, and an initial state, and produces a new function that takes sequences of inputs to sequences of outputs.

```
1 mealy :: (Circ s i o) -> s -> [i] -> [o]
2 mealy _ s []     = (s, [])
3 mealy c s (i:is) = (sf, o:os)
4   where (s', o)  = c s i
5         (sf, os) = mealy c s' is
```

For example, running the adder on the following input sequence

```
1 is = [Just (1, 3), Nothing, Just (0,1), Just (1,2), Nothing]
```

and initial state `Nothing`, gives the following outputs:

```
1 mealy add  Nothing is = [Nothing, Just 4, Just 1, Nothing, Just 3]
```

Here, an output of `Nothing` means that no new output is available, while a `Just` value indicates a valid output. When translating to hardware, the difference between `Just` and `Nothing` is encoded into an additional wire that serves as ready bit. Note that the output timing of the adder depends on its inputs. As such, the adder leaks information about its inputs via its timing. We want to capture

this leakage with a specification. This specification should allow a client of the adder to use it securely, without leaking any secrets it may operate on.

Adder Specification. For this, we first start by giving an input/output specification to the adder. Rather than operating over optional `Maybe` types, which encode whether inputs or outputs are ready at the hardware level, we want the specification to operate at a higher level – as a pure function over plain integers. We give this specification via the function `spec` below.

```
spec :: (Int, Int) -> Int
spec (a, b) = a + b
```

Observation Function. Next, we want to specify the leakage of the circuit at the same level of abstraction. Note that the adder executes faster if the first input is `0` and slower, otherwise. An attacker that can observe the timing of the circuit can use this difference to make inference about the values we have been operating on. We can formally specify a timing attacker via an observation function `obs` over the outputs of the circuit.

```
isJust :: Maybe a -> Bool
isJust (Just _) = True
isJust _        = False

obs :: Maybe Int -> Bool
obs = isJust
```

Leakage Description. We capture this in the following leakage description, which returns a Boolean, indicating whether or not the circuit took the fast path.

```
leak :: (Int, Int) -> Bool
leak (0, _) = True
leak _      = False
```

Using the Leakage Description. A client of the adder may now use this description to program the adder securely. In particular, the client needs to ensure that the output of function `leak` stays the same, no matter which secret value the computation operates on. Indeed, we can think of `add` as a simple processor offering a single instruction. When checking whether programs for the processor are secure, we may now use `spec` to describe its functional behavior, and `leak` to describe its leakage. This allows us to completely avoid reasoning about the more complex hardware, which makes verification much easier. But, to do this soundly, we must prove both leakage and functional correctness specification correct.

2.2 Verifying the Leakage Description

Driver.

```
driver :: (Int, Int) -> [Maybe (Int, Int)]
driver (0, b) = [Just (0,b)]
driver (a, b) = [Just (a,b), Nothing]
```

Inversion Function.

```
invert :: Bool -> (Int, Int)
invert True = (0,0)
invert False = (1,0)
```

Circuit Combinators.

```
seqC :: Circ s1 i io -> Circ s2 io o -> Circ (s1,s2) i o
seqC c1 c2 (s1,s2) i  = ((s1', s2'), o)
  where
    (s1', i2) = c1 s1 i
    (s2', o)  = c2 s2 i2

prod :: Circ s1 i [io] -> Circ s2 io o -> Circ (s1,s2) i [o]
prod c1 c2 (s1,s2) i = ((s1',s2'), os)
  where
    (s1', is) = c1 s1 i
    (s2', os) = mealy c2 s2 is

lift :: (i->o) -> (() -> i -> ((), o))
lift f _ i  = ((), f i)
```

Combined Circuits.

```
type ImpObsState = ((), (Maybe Int, ()))

impObs :: ImpObsState -> (Int, Int) -> (ImpObsState, [Bool])
impObs = (lift driver) `prod` (add `seqC` (lift obs))

type LeakSimState = ((((),()),()), (Maybe Int, ()))
leakSim :: Circ LeakSimState (Int, Int) [Bool]
leakSim = ((lift leak) `seqC` (lift invert) `seqC` (lift driver)) `prod`
    (add `seqC` (lift obs))
```

Projection Function.

```
proj :: ImpObsState -> LeakSimState
proj ((), (Just _,  ())) = toLeakSim (Just 1)
proj ((), (Nothing, ())) = toLeakSim Nothing
```

Checking Leakage.

```
{-# ANN proj_commutes (Theory Base.axioms) #-}
proj_commutes :: ImpObsState -> (Int, Int) -> Bool
proj_commutes s i = proj si == sl && eqList oi ol
    where
       (si, oi) = impObs s i
       (sl, ol) = leakSim (proj s) i
```

Checking Correctness: Invariant.

```
invariant :: ImpState -> Bool
invariant ((), Nothing) = True
invariant ((), Just _)  = False

{-# ANN invariant_init (Theory Base.axioms) #-}
invariant_init :: Bool
invariant_init = invariant initImp
```

```
{-# ANN invariant_cons (Theory Base.axioms) #-}
invariant_cons :: ImpState -> (Int, Int) -> Bool
invariant_cons s i = invariant s --> invariant s'
    where
      (s', _) = imp s i
```

Checking Correctness: Outputs.

```
getRes :: [Maybe Int] -> Int
getRes [Nothing, Just x] = x
getRes [Just x]          = x

{-# ANN output_correct (Theory Base.axioms) #-}
output_correct :: ImpState -> (Int, Int) -> Bool
output_correct s i = invariant s --> getRes oi == os
    where
        (_, oi) = imp s i
        os      = spec i
```

2.3 Example: Processor

3 Related Work

LeaVe [1] builds leakage models at the ISA level, but relies on the assumption that the processor is functionally correct. Unlike Sys, LeaVe does not produce an executable artifact for the high-level leakage.

References

[1] Zilong Wang, Gideon Mohr, Klaus von Gleissenthall, Jan Reineke, and Marco Guarnieri. 2023. Specification and Verification of Side-channel Security for Open-source Processors via Leakage Contracts. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security* (Copenhagen, Denmark) *(CCS '23)*. Association for Computing Machinery, New York, NY, USA, 2128–2142. https://doi.org/10.1145/3576915.3623192